

QECCertificates: certified code parameters and fault distances for quantum error correction

Shuoming An*

Draft of 30 September 2026

Abstract

Code parameters are found by search, and a search ends in a solver’s verdict. Where the verdict is trusted, a reader who wants to check it has to re-run the search. **QECCertificates** is a Lean 4 library and, in its next version, a certificate toolchain that replaces that verdict with an object small enough to check: a proof that a distance is at least a claimed value, or a witness that it is at most one, checked by a kernel that knows nothing about the search. This paper describes what the library contains, what is machine-checked about it, how two companion developments use it as their shared core, and what it deliberately does not do.

1 Why this exists

Quantum low-density parity-check codes are designed by search: choose a construction, enumerate candidates, compute a distance. The public schema behind the qLDPC Challenge records, in its own words, what that costs. A distance that is not certified is reported as an upper bound – for a non-CSS code because the Pauli-weight certifier “is not available yet”, for a circuit-level distance because an exact tier “is deferred future work” – and what lies in between is described as evidence but not as a proof.¹

The nearest prior work on the kernel side, Lean-QEC [1], reduces the distance question to satisfiability and replays the solver’s proof inside a proof assistant. Its reported reach is 90 qubits, and its authors state that at 144 qubits the replay itself is the wall. That line of work is closest to ours in method, and it is also why we treat the encoding step as the interesting one: what makes a certified bound useful is not that a solver ran, but that the object closing the argument is checkable by something that did not run the solver.

2 What is in the library

The library is a Lean 4 development of 55 modules, built against mathlib at a pinned revision, on the same toolchain (Lean 4.34) as the two developments that consume it.

- **A $\text{GF}(2)$ layer.** Vectors over $\mathbb{Z}_2$, row spaces, a trusted row reduction with a proved pivot invariant, kernel bases, membership decisions that are computable, weight-limited enumeration

*Guangdong Provincial Key Laboratory of Computility Microelectronics. Correspondence: <https://github.com/QCL-SUAT/QECCertificates>.

¹The quotations are from the schema’s own descriptions of the distance tiers, as read on 30 September 2026; the wording is theirs, not ours.

with a covering theorem, and a certificate for a distance lower bound as a single decidable predicate.

- **An operator layer.** Pauli expressions with a symplectic representation, and translation theorems that make the vector-level decisions equal to statements about the matrices the surrounding ecosystem uses.
- **A certificate framework.** An in-kernel checker for LRAT proofs [2], with soundness and completeness proved about the checker itself; a formalization of the encoding step that ties satisfiability of the encoded formula to the existence of a light logical operator, in both directions; and symmetry breaking by lex-leader predicates, with the faithfulness of the reduction proved rather than assumed.
- **Instances.** Kernel-checked assertions of the triple (n, k, d) for a table of codes, including a hypergraph-product family covered by general theorems rather than by instance-by-instance evaluation.

3 What is checked, and how

Two properties are meant to be machine-checked rather than asserted, because both are the kind of claim that is either checked or is marketing.

The trusted base. No declaration in the library uses `sorry`, a custom axiom, or `native_decide`; computations that matter are closed by kernel reduction. The root module carries an audit region of 1,060 lines, each a request that the kernel print the axioms a load-bearing declaration depends on; a script in `tools/` reads those lines out of a build log and fails on any axiom outside `propext`, `Classical.choice` and `Quot.sound`. The same script fails when the log contains no output at all: a fresh clone that never built is not a pass.

The build. Continuous integration builds the library with the prebuilt mathlib cache, requires zero error lines and zero warnings attributed to this package’s own files, and then runs the audit above on the log. Warnings from dependencies are not silenced; they are simply not this package’s.

4 How the two companion developments use it

The library is not a standalone artifact. The GF(2), Pauli and shared code modules were merged, file by file, from two companion developments whose copies of that layer had drifted apart; the merge took the fuller of each pair, rewrote no proof, and the two developments now import the package instead of carrying copies. One companion builds an interface layer in which the row space defined here is proved equal to the row space of Lean-QEC, the symplectic predicate is proved equivalent to commutation of actual Pauli matrices, and a state on the logical register is bounded by the logical dimension proved here. The other builds fault complexes, gauged instantiations and a condition layer for logical measurement on top of the same core. Both pin the package by commit; the papers that report those results cite this library as the home of the shared statements, so a reader can check the interface claims without reading either paper’s repository.

5 Using it

```
# in a lakefile.toml
[[require]]
name = "QECCertificates"
git = "https://github.com/QCL-SUAT/QECCertificates.git"
rev = "<40-hex commit>"

lake exe cache get    # prebuilt mathlib, on a machine that has not built it
lake build
```

Nothing is configured per machine: every dependency is fetched at a pinned revision, and a script that links a local prebuilt layer is optional and never required for a build to succeed. The audit region is part of the build, so a downstream user sees the axiom list for the statements they consume in their own build log.

6 What it does not do

The library does not simulate circuits, decode syndromes, or enumerate codes; those are served by mature projects, and the point of a certificate layer is to sit beside them rather than to reimplement them. It does not claim a general decision procedure for distances: what it offers is a format in which a bound, once found, is checkable, and a kernel that decides the check. It does not accept a solver’s word at any step: the in-kernel path closes the argument with a proof object, and the certificate toolchain that lands next is written so that the checker is independent of the solver that produced the proof. Finally, it does not carry the code tables of the field: a handful of instances are included because they are load-bearing for the two companion developments, not as a database.

7 Roadmap

Version 0.1 is the formal layer described here, released under Apache-2.0 and pinned by commit. The next version adds the Python certificate chain: the CNF encoder for distance lower bounds with a known-value gate, LRAT/RUP checkers written twice in different languages so that a bug in one is unlikely to be mirrored in the other, and the adapters that turn a check matrix into a certified bound. A white paper beyond this draft will report the toolchain’s end-to-end results on the instances the two companion developments use.

References

- [1] M. Ehatamm, Y. Lee, X. Wu and R. Tao, *End-to-End Formalization of Quantum Error Correction*, arXiv:2605.16523 (2026).
- [2] L. Cruz-Filipe, M. J. H. Heule, W. A. Hunt, M. Kaufmann and P. Schneider-Kamp, *Efficient certified RAT verification*, in Automated Deduction (CADE-26), LNCS 10395, 220–236 (2017).
- [3] The qLDPC Challenge, public data schema and tier descriptions, <https://github.com/unitaryfoundation/qldpc-challenge> (read 30 September 2026).